

```
pip install numpy pillow matplotlib scipy pandas
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: pillow in /usr/local/lib/python3.13/dist-packages (11.3.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages (3.10.0)
Requirement already satisfied: scipy in /usr/local/lib/python3.13/dist-packages (1.16.3)
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cyclery>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (26.3)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas) (2026.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

```
import os
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.fft import fft2, fftshift

warnings.filterwarnings("ignore")

plt.rcParams.update({
    "font.family": "monospace",
    "figure.titlesize": 13,
    "axes.titlesize": 11,
    "figure.dpi": 150
})

print("Cell 1: Environment and libraries successfully loaded.")
```

Cell 1: Environment and libraries successfully loaded.

```
NUM_SEEDS = 100
GRID_HEIGHT = 1024
GRID_WIDTH = 2048

LOW_BAND_CUTOFF = 0.15
MID_BAND_CUTOFF = 0.40

OPERATORS = {
    "AND":      lambda X, Y: X & Y,
    "OR":       lambda X, Y: X | Y,
    "XOR":      lambda X, Y: X ^ Y,
    "ADD":      lambda X, Y: X + Y,
    "MULTIPLY": lambda X, Y: np.bitwise_and(X * Y, 0x7FFFFFFF),
    "HYBRID":   lambda X, Y: (X - Y) ^ (X + Y)
}

OPERATOR_LABELS = {
    "AND":      "AND  (x & y)",
    "OR":       "OR   (x | y)",
    "XOR":      "XOR  (x ^ y)",
    "ADD":      "ADD  (x + y)",
    "MULTIPLY": "MULTIPLY (x * y)",
    "HYBRID":   "HYBRID (x-y)⊕(x+y)"
}

print(f"Cell 2: Configured {len(OPERATORS)} operators for {GRID_WIDTH}x{GRID_HEIGHT} simulation.")
```

Cell 2: Configured 6 operators for 2048x1024 simulation.

```
def generate_luminance_field(op_func, seed_x: int, seed_y: int, seed_scale: int,
                             h: int = GRID_HEIGHT, w: int = GRID_WIDTH) -> np.ndarray:
    """
    Simulates the core Bit Dot Night discrete coordinate mapping and cosine luminance field.
    f(x, y) = 0(x + sx, y + sy)
    L(x, y) = 0.5 * (1.0 + cos(f(x, y) * k)) normalized to [0, 1]
    """
    ys, xs = np.indices((h, w), dtype=np.int64)
    X = xs + seed_x
    Y = ys + seed_y

    O = op_func(X, Y)

    L = 0.5 * (1.0 + np.cos(O * (seed_scale * 1e-6)))

    return L.astype(np.float32)

print("Cell 3: Generative luminance simulation engine compiled.")
```

Cell 3: Generative luminance simulation engine compiled.

```
def compute_metrics(L: np.ndarray):
    """
    Evaluates Brightness Variance, Structural Coherence, and 2D FFT Radial Energy Partitioning.
    """
    h, w = L.shape

    # 1. Brightness
    var_val = float(np.var(L))

    # 2. Structural Coherence
    diff_h = np.abs(L[:, :-1] - L[:, 1:])
    diff_v = np.abs(L[:, -1, :] - L[1:, :])
    min_h, min_w = min(diff_h.shape[0], diff_v.shape[0]), min(diff_h.shape[1], diff_v.shape[1])
    coherence = float(0.5 * (diff_h[:min_h, :min_w].mean() + diff_v[:min_h, :min_w].mean()))

    # 3. 2D Spectral Energy Bands via FFT
    F = fftshift(fft2(L))
```

```
mag_sq = np.abs(F) ** 2
total_energy = mag_sq.sum()

cy, cx = h // 2, w // 2
ys = (np.arange(h) - cy) / cy
xs = (np.arange(w) - cx) / cx
XX, YY = np.meshgrid(xs, ys)
R = np.sqrt(XX**2 + YY**2)

e_low = float(mag_sq[R <= LOW_BAND_CUTOFF].sum())
e_mid = float(mag_sq[(R > LOW_BAND_CUTOFF) & (R <= MID_BAND_CUTOFF)].sum())
e_high = float(mag_sq[R > MID_BAND_CUTOFF].sum())

low_pct = (e_low / total_energy) * 100.0 if total_energy > 0 else 0.0
mid_pct = (e_mid / total_energy) * 100.0 if total_energy > 0 else 0.0
high_pct = (e_high / total_energy) * 100.0 if total_energy > 0 else 0.0

return var_val, low_pct, mid_pct, high_pct, coherence, mag_sq

print("Cell 4: Metric evaluation functions compiled.")
```

Cell 4: Metric evaluation functions compiled.

```
np.random.seed(42)

summary_results = []
sample_spectra = {}
sample_gradients = {}

print("=" * 76)
print(f" Running Monte Carlo Ensemble: {NUM_SEEDS} Seeds per Operator ({GRID_WIDTH}x{GRID_HEIGHT})")
print("=" * 76)

for name, func in OPERATORS.items():
    var_list = []
    low_list = []
    mid_list = []
    high_list = []
    coh_list = []

    for i in range(NUM_SEEDS):
        sx = int(np.random.randint(0, 10000))
        sy = int(np.random.randint(0, 10000))
        scale = int(np.random.randint(100, 1000000))

        L = generate_luminance_field(func, sx, sy, scale)
        var, lp, mp, hp, coh, mag = compute_metrics(L)

        var_list.append(var)
        low_list.append(lp)
        mid_list.append(mp)
        high_list.append(hp)
        coh_list.append(coh)

        if i == 0:
            sample_spectra[name] = mag
            diff_h = np.abs(L[:, :-1] - L[:, 1:])
            diff_v = np.abs(L[:-1, :] - L[1:, :])
            min_h, min_w = min(diff_h.shape[0], diff_v.shape[0]), min(diff_h.shape[1], diff_v.shape[1])
            sample_gradients[name] = 0.5 * (diff_h[:min_h, :min_w] + diff_v[:min_h, :min_w])

    summary_results.append({
        "Operator": OPERATOR_LABELS[name],
        "Variance  $\sigma^2$ ": round(float(np.mean(var_list)), 5),
        "E_low (%)": round(float(np.mean(low_list)), 1),
        "E_mid (%)": round(float(np.mean(mid_list)), 1),
        "E_high (%)": round(float(np.mean(high_list)), 1),
        "Coherence C": round(float(np.mean(coh_list)), 5),
    })

    print(f"✓ Completed {name:<10} |  $\sigma^2$  = {np.mean(var_list):.5f} | C = {np.mean(coh_list):.5f}")

df_results = pd.DataFrame(summary_results)
df_results.to_csv("bit_dot_night_quant_results.csv", index=False)

print("\n" + "=" * 76)
print(" TABLE 1 VALUES (PASTE INTO LATEX)")
print("=" * 76)
print(df_results.to_string(index=False))
print("=" * 76)
print("CSV exported: 'bit_dot_night_quant_results.csv'")
```

```
=====
Running Monte Carlo Ensemble: 100 Seeds per Operator (2048x1024)
=====
✓ Completed AND      |  $\sigma^2$  = 0.12749 | C = 0.14772
✓ Completed OR       |  $\sigma^2$  = 0.12424 | C = 0.15043
✓ Completed XOR      |  $\sigma^2$  = 0.12502 | C = 0.24003
✓ Completed ADD      |  $\sigma^2$  = 0.12385 | C = 0.16706
✓ Completed MULTIPLY |  $\sigma^2$  = 0.12501 | C = 0.40539
✓ Completed HYBRID   |  $\sigma^2$  = 0.12500 | C = 0.24695

=====
TABLE 1 VALUES (PASTE INTO LATEX)
=====
Operator  Variance  $\sigma^2$   E_low (%)  E_mid (%)  E_high (%)  Coherence C
AND (x & y)  0.12749      81.5      11.3      7.2      0.14772
OR (x | y)   0.12424      80.6      11.8      7.6      0.15043
XOR (x ^ y)  0.12502      74.3      14.0      11.7     0.24003
ADD (x + y)  0.12385      76.0      20.8      3.2      0.16706
MULTIPLY (x * y) 0.12501      67.3      3.6      29.1     0.40539
HYBRID (x-y)⊕(x+y) 0.12500      72.2      7.7      20.1     0.24695
=====
CSV exported: 'bit_dot_night_quant_results.csv'
```

```
# Figure 1
fig1, axes1 = plt.subplots(2, 3, figsize=(14, 7), facecolor="black")
fig1.suptitle("2D FFT Magnitude Spectra (log-Scaled)", fontsize=14, fontweight="bold", color="white", y=0.98)
```



```
for ax, name in zip(axes1.flat, OPERATORS.keys()):
    if name in sample_spectra:
        mag = sample_spectra[name]
        im = ax.imshow(np.log1p(mag), cmap="magma", aspect="auto")
        ax.set_title(OPERATOR_LABELS[name], fontsize=11, fontweight="bold", color="white", pad=8)
    ax.axis("off")

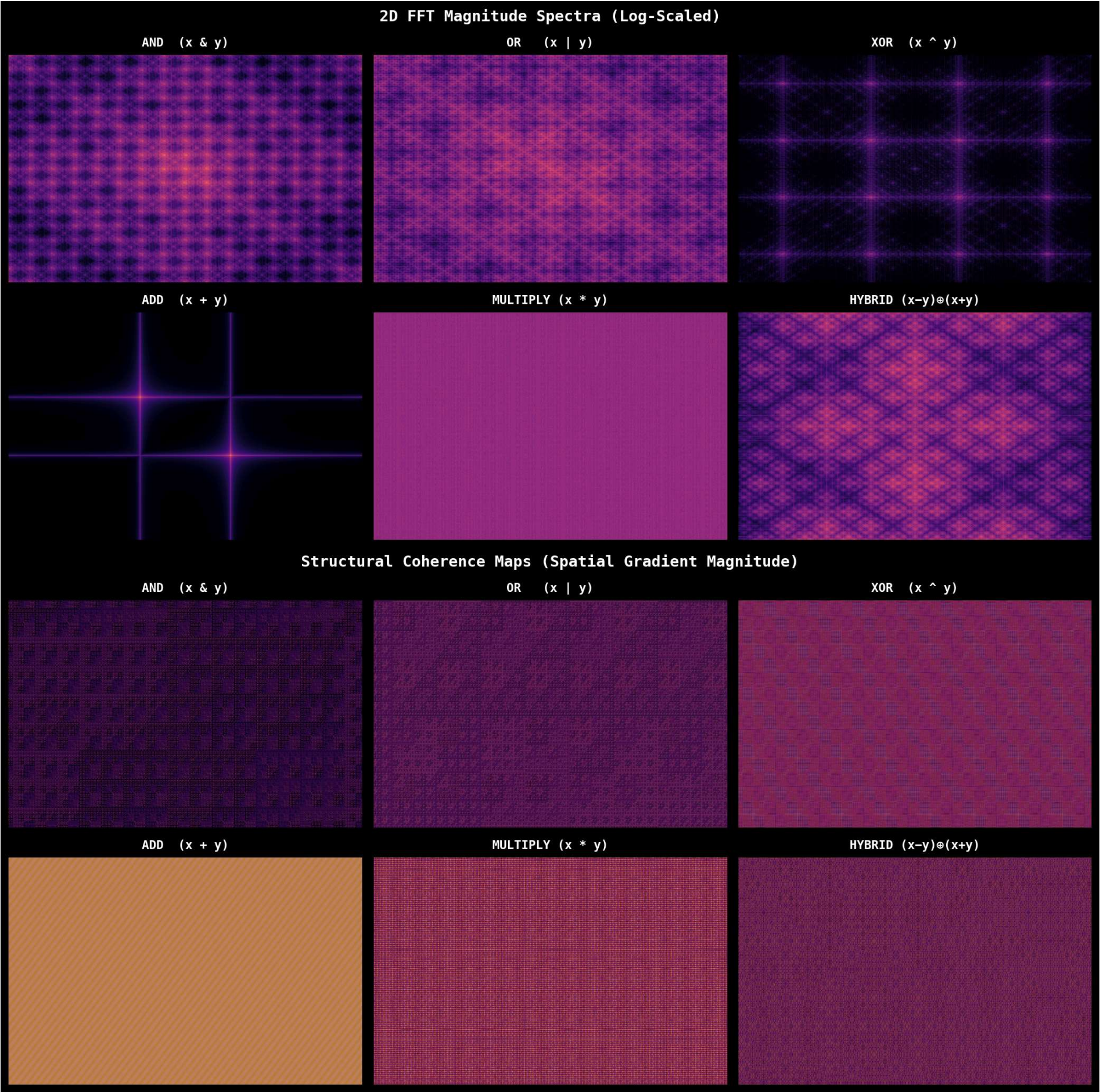
plt.tight_layout()
plt.savefig("fft_spectra.png", dpi=300, bbox_inches="tight", facecolor="black")
plt.show()

# Figure 2
fig2, axes2 = plt.subplots(2, 3, figsize=(14, 7), facecolor="black")
fig2.suptitle("Structural Coherence Maps (Spatial Gradient Magnitude)", fontsize=14, fontweight="bold", color="white", y=0.98)

for ax, name in zip(axes2.flat, OPERATORS.keys()):
    if name in sample_gradients:
        grad = sample_gradients[name]
        im = ax.imshow(grad, cmap="inferno", aspect="auto")
        ax.set_title(OPERATOR_LABELS[name], fontsize=11, fontweight="bold", color="white", pad=8)
    ax.axis("off")

plt.tight_layout()
plt.savefig("coherence_maps.png", dpi=300, bbox_inches="tight", facecolor="black")
plt.show()

print("Figures successfully generated and exported as 'fft_spectra.png' and 'coherence_maps.png'")
```



Figures successfully generated and exported as 'fft_spectra.png' and 'coherence_maps.png'

```
from scipy.stats import linregress

def spatial_shannon_entropy(L: np.ndarray, bins: int = 64) -> float:
    """
    Computes 1st-order Shannon Entropy H(L) = -sum(p * log2(p)).
    Measures information density and tonal uncertainty.
    """
```



```
hist, _ = np.histogram(L, bins=bins, range=(0.0, 1.0), density=True)
p = hist / np.sum(hist)
p = p[p > 0]
return float(-np.sum(p * np.log2(p)))

def spectral_decay_exponent(mag_sq: np.ndarray) -> float:
    """
    Fits Power-law decay: S(f) ~ 1 / f^beta.
    beta ~ 0: White noise (high-frequency random)
    beta ~ 1: 1/f Pink noise (natural structures, architectural facades)
    beta ~ 2: Brownian/Gaussian diffusion (smooth gradients)
    """
    h, w = mag_sq.shape
    cy, cx = h // 2, w // 2
    ys, xs = np.indices((h, w))
    r = np.sqrt(((xs - cx) / cx)**2 + ((ys - cy) / cy)**2)

    mask = (r > 0.02) & (r <= 0.8)
    r_vals = r[mask]
    p_vals = mag_sq[mask]

    log_r = np.log10(r_vals)
    log_p = np.log10(p_vals + 1e-12)

    slope, _, _, _ = linregress(log_r, log_p)
    return float(-slope)

def morans_spatial_autocorrelation(L: np.ndarray, subsample_step: int = 8) -> float:
    """
    Computes Moran's I for spatial spatial clustering and autocorrelation.
    I > 0: Clustered/Structured, I = 0: Random, I < 0: Dispersed/Checkerboard.
    """
    sub = L[:,::subsample_step, ::subsample_step]
    h, w = sub.shape
    z = sub - np.mean(sub)
    s0 = np.sum(z**2)
    if s0 == 0:
        return 0.0

    lag = np.zeros_like(z)
    lag[1:, :] += z[:-1, :]
    lag[:-1, :] += z[1:, :]
    lag[:, 1:] += z[:, :-1]
    lag[:, :-1] += z[:, 1:]

    I = (h * w / (4 * h * w)) * (np.sum(z * lag) / s0)
    return float(I)

print("Cell 7: Advanced mathematical metrics engine loaded.")
```

Cell 7: Advanced mathematical metrics engine loaded.

```
from scipy.stats import f_oneway

np.random.seed(42)

advanced_records = []
operator_entropy_samples = {name: [] for name in OPERATORS.keys()}
operator_beta_samples = {name: [] for name in OPERATORS.keys()}
operator_moran_samples = {name: [] for name in OPERATORS.keys()}

print("=" * 78)
print(f" Computing Advanced Metrics over {NUM_SEEDS} Seeds per Operator")
print("=" * 78)

for name, func in OPERATORS.items():
    ent_list = []
    beta_list = []
    moran_list = []

    for i in range(NUM_SEEDS):
        sx = int(np.random.randint(0, 10000))
        sy = int(np.random.randint(0, 10000))
        scale = int(np.random.randint(100, 1000000))

        L = generate_luminance_field(func, sx, sy, scale)

        # 1. Shannon Entropy
        h_val = spatial_shannon_entropy(L)
        ent_list.append(h_val)
        operator_entropy_samples[name].append(h_val)

        # 2. Spectral Power Decay (Beta)
        F = fftshift(fft2(L))
        mag_sq = np.abs(F) ** 2
        beta_val = spectral_decay_exponent(mag_sq)
        beta_list.append(beta_val)
        operator_beta_samples[name].append(beta_val)

        # 3. Moran's I (Spatial Clustering)
        moran_val = morans_spatial_autocorrelation(L)
        moran_list.append(moran_val)
        operator_moran_samples[name].append(moran_val)

    advanced_records.append({
        "Operator": OPERATOR_LABELS[name],
        "Shannon Entropy H (bits)": round(float(np.mean(ent_list)), 4),
        "Entropy Std": round(float(np.std(ent_list)), 4),
        "Spectral Decay (β)": round(float(np.mean(beta_list)), 3),
        "Moran's I (Clustering)": round(float(np.mean(moran_list)), 4),
    })

print(f"✓ {name:<10} | H = {np.mean(ent_list):.4f} bits | β = {np.mean(beta_list):.3f} | Moran's I = {np.mean(moran_list):.4f}")

df_adv = pd.DataFrame(advanced_records)
df_adv.to_csv("bit_dot_night_advanced_metrics.csv", index=False)
```

```
# One-Way ANOVA Statistical Hypothesis Testing
f_stat_ent, p_val_ent = f_oneway(*operator_entropy_samples.values())
f_stat_beta, p_val_beta = f_oneway(*operator_beta_samples.values())
f_stat_moran, p_val_moran = f_oneway(*operator_moran_samples.values())

print("\n" + "=" * 78)
print("  ADVANCED METRIC RESULTS (TABLE 2 EXTENSION)")
print("=" * 78)
print(df_adv.to_string(index=False))
print("=" * 78)

print("\n" + "=" * 78)
print("  ONE-WAY ANOVA STATISTICAL SIGNIFICANCE (p-values)")
print("=" * 78)
print(f"Shannon Entropy (H) : F-statistic = {f_stat_ent:.3f},  p-value = {p_val_ent:.4e}")
print(f"Spectral Decay (β) : F-statistic = {f_stat_beta:.3f},  p-value = {p_val_beta:.4e}")
print(f"Moran's I           : F-statistic = {f_stat_moran:.3f},  p-value = {p_val_moran:.4e}")
print("=" * 78)
```

```
=====
Computing Advanced Metrics over 100 Seeds per Operator
=====
✓ AND      | H = 5.4812 bits | β = 2.305 | Moran's I = 0.3138
✓ OR       | H = 5.5249 bits | β = 2.301 | Moran's I = 0.4342
✓ XOR      | H = 5.7104 bits | β = 1.576 | Moran's I = 0.0950
✓ ADD      | H = 5.6910 bits | β = 2.738 | Moran's I = 0.1765
✓ MULTIPLY | H = 5.7223 bits | β = -0.006 | Moran's I = -0.0019
✓ HYBRID   | H = 5.7009 bits | β = 1.889 | Moran's I = 0.3122

=====
ADVANCED METRIC RESULTS (TABLE 2 EXTENSION)
=====
Operator  Shannon Entropy H (bits)  Entropy Std  Spectral Decay (β)  Moran's I (Clustering)
AND (x & y)      5.4812      0.2689      2.305              0.3138
OR (x | y)       5.5249      0.1938      2.301              0.4342
XOR (x ^ y)      5.7104      0.0697      1.576              0.0950
ADD (x + y)      5.6910      0.2409      2.738              0.1765
MULTIPLY (x * y) 5.7223      0.0006      -0.006            -0.0019
HYBRID (x-y)⊕(x+y) 5.7009      0.0783      1.889              0.3122
=====

=====
ONE-WAY ANOVA STATISTICAL SIGNIFICANCE (p-values)
=====
Shannon Entropy (H) : F-statistic = 37.497,  p-value = 1.9168e-33
Spectral Decay (β)  : F-statistic = 155.410,  p-value = 2.1366e-105
Moran's I           : F-statistic = 17.494,  p-value = 3.5214e-16
=====
```

```
# 1. Distribution Boxplots (H, Beta, Moran's I)
fig, axes = plt.subplots(1, 3, figsize=(16, 5), facecolor="white")
fig.suptitle("Statistical Distributions Across Operators (N=100 Seeds per Operator)", fontsize=13, fontweight="bold", y=1.02)

# Shannon Entropy
data_h = [operator_entropy_samples[op] for op in OPERATORS.keys()]
axes[0].boxplot(data_h, labels=list(OPERATORS.keys()), patch_artist=True,
                boxprops=dict(facecolor="#6a11cb", color="black", alpha=0.7),
                medianprops=dict(color="yellow", linewidth=1.5))
axes[0].set_title("Shannon Entropy (H in bits)", fontweight="bold")
axes[0].set_ylabel("Bits")
axes[0].grid(axis="y", linestyle="--", alpha=0.5)

# Spectral Decay (Beta)
data_beta = [operator_beta_samples[op] for op in OPERATORS.keys()]
axes[1].boxplot(data_beta, labels=list(OPERATORS.keys()), patch_artist=True,
                boxprops=dict(facecolor="#2575fc", color="black", alpha=0.7),
                medianprops=dict(color="yellow", linewidth=1.5))
axes[1].set_title("Spectral Decay Exponent (β)", fontweight="bold")
axes[1].set_ylabel("Power-Law Slope β")
axes[1].grid(axis="y", linestyle="--", alpha=0.5)

# Moran's I
data_moran = [operator_moran_samples[op] for op in OPERATORS.keys()]
axes[2].boxplot(data_moran, labels=list(OPERATORS.keys()), patch_artist=True,
                boxprops=dict(facecolor="#f87171", color="black", alpha=0.7),
                medianprops=dict(color="yellow", linewidth=1.5))
axes[2].set_title("Moran's I (Spatial Autocorrelation)", fontweight="bold")
axes[2].set_ylabel("Spatial Lag Correlation")
axes[2].grid(axis="y", linestyle="--", alpha=0.5)

plt.tight_layout()
plt.savefig("metric_distributions.png", dpi=300, bbox_inches="tight")
plt.show()

# 2. Comparative Radar Fingerprint Profile
categories = ['Variance (σ²)', 'High Freq %', 'Entropy (H)', 'Fractal β', "Moran's I"]
num_vars = len(categories)

# Normalize metrics between [0.1, 1.0] for radar scaling
def min_max_norm(arr):
    mn, mx = np.min(arr), np.max(arr)
    return (arr - mn) / (mx - mn + 1e-8) * 0.8 + 0.2

mean_vars = [df_results.loc[df_results['Operator'] == OPERATOR_LABELS[op], 'Variance σ²'].values[0] for op in OPERATORS.keys()]
mean_high = [df_results.loc[df_results['Operator'] == OPERATOR_LABELS[op], 'E_high (%)'].values[0] for op in OPERATORS.keys()]
mean_h     = [df_adv.loc[df_adv['Operator'] == OPERATOR_LABELS[op], 'Shannon Entropy H (bits)'].values[0] for op in OPERATORS.keys()]
mean_beta  = [df_adv.loc[df_adv['Operator'] == OPERATOR_LABELS[op], 'Spectral Decay (β)'].values[0] for op in OPERATORS.keys()]
mean_i     = [df_adv.loc[df_adv['Operator'] == OPERATOR_LABELS[op], "Moran's I (Clustering)"].values[0] for op in OPERATORS.keys()]

norm_matrix = np.column_stack([
    min_max_norm(mean_vars),
    min_max_norm(mean_high),
    min_max_norm(mean_h),
    min_max_norm(mean_beta),
    min_max_norm(mean_i)
])

angles = np.linspace(0, 2 * np.pi, num_vars, endpoint=False).tolist()
angles += angles[:1]
```

```
fig, ax = plt.subplots(figsize=(7, 7), subplot_kw=dict(polar=True), facecolor="white")
ax.set_theta_offset(np.pi / 2)
ax.set_theta_direction(-1)
plt.xticks(angles[:-1], categories, fontweight="bold", size=10)

colors = ["#e41a1c", "#377eb8", "#4daf4a", "#984ea3", "#ff7f00", "#a65628"]

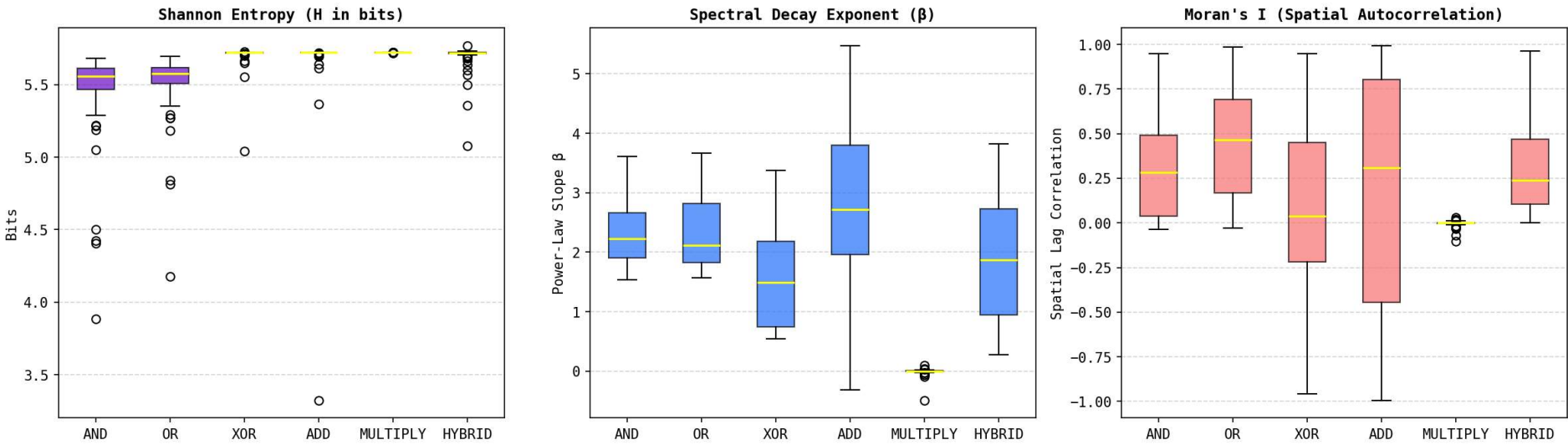
for idx, (op, color) in enumerate(zip(OPERATORS.keys(), colors)):
    values = norm_matrix[idx].tolist()
    values += values[:1]
    ax.plot(angles, values, color=color, linewidth=2, label=op)
    ax.fill(angles, values, color=color, alpha=0.1)

ax.set_rlabel_position(0)
plt.yticks([0.2, 0.4, 0.6, 0.8, 1.0], ["0.2", "0.4", "0.6", "0.8", "1.0"], color="grey", size=8)
plt.ylim(0, 1.05)
plt.title("Operator Aesthetic & Structural Profiles", y=1.08, fontweight="bold", fontsize=12)
plt.legend(loc='upper right', bbox_to_anchor=(1.25, 1.1), frameon=True)

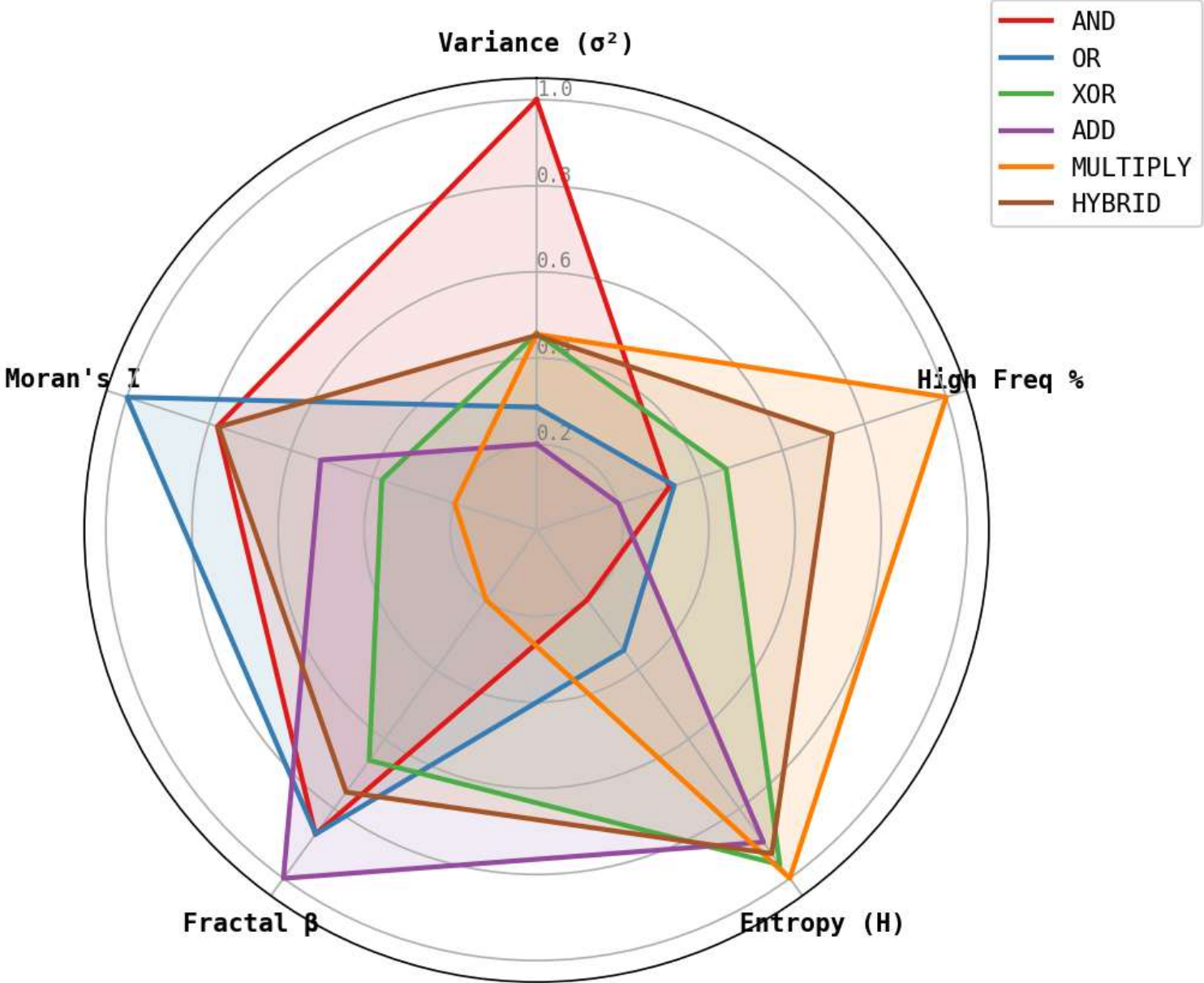
plt.tight_layout()
plt.savefig("operator_radar_profile.png", dpi=300, bbox_inches="tight")
plt.show()

print("Publication figures saved: 'metric_distributions.png' and 'operator_radar_profile.png'")
```

Statistical Distributions Across Operators (N=100 Seeds per Operator)



Operator Aesthetic & Structural Profiles



Publication figures saved: 'metric_distributions.png' and 'operator_radar_profile.png'

```
deltas = np.arange(1, 32)
sensitivity_curves = {name: [] for name in OPERATORS.keys()}

for name, func in OPERATORS.items():
    div_per_delta = []
    for d in deltas:
        diffs = []
        for _ in range(20):
            sx, sy = np.random.randint(0, 5000), np.random.randint(0, 5000)
            scale = np.random.randint(100, 1000000)
            L1 = generate_luminance_field(func, sx, sy, scale, h=256, w=512)
            L2 = generate_luminance_field(func, sx + d, sy + d, scale, h=256, w=512)
```

```

        generate_luminance_image(image, dx + 0.5, dy + 0.5, state, h=256, w=256)
        diffs.append(np.mean(np.abs(L1 - L2)))
    div_per_delta.append(np.mean(diffs))
    sensitivity_curves[name] = div_per_delta

def get_1d_radial_psd(mag_sq, num_bins=60):
    h, w = mag_sq.shape
    cy, cx = h // 2, w // 2
    ys, xs = np.indices((h, w))
    r = np.sqrt(((xs - cx) / cx)**2 + ((ys - cy) / cy)**2)

    r_flat = r.flatten()
    p_flat = mag_sq.flatten()

    bin_edges = np.linspace(0.02, 0.8, num_bins + 1)
    bin_centers = 0.5 * (bin_edges[:-1] + bin_edges[1:])
    radial_prof = np.zeros(num_bins)

    for i in range(num_bins):
        mask = (r_flat >= bin_edges[i]) & (r_flat < bin_edges[i+1])
        if np.any(mask):
            radial_prof[i] = np.mean(p_flat[mask])
        else:
            radial_prof[i] = 1e-12
    return bin_centers, radial_prof

fig = plt.figure(figsize=(18, 5), facecolor="white")

ax1 = fig.add_subplot(1, 3, 1)
for name, curve in sensitivity_curves.items():
    ax1.plot(deltas, curve, marker='o', markersize=3, label=name)
ax1.set_title("Seed Perturbation Divergence S( $\delta$ )", fontweight="bold")
ax1.set_xlabel("Seed Perturbation Offset  $\delta$ ")
ax1.set_ylabel("Mean Absolute Difference E[ $|\Delta L|$ ]")
ax1.grid(True, linestyle="--", alpha=0.5)
ax1.legend(fontsize=8)

ax2 = fig.add_subplot(1, 3, 2)
for name in OPERATORS.keys():
    freqs, psd = get_1d_radial_psd(sample_spectra[name])
    ax2.plot(np.log10(freqs), np.log10(psd + 1e-12), label=name)

ref_x = np.linspace(-1.5, -0.1, 50)
ax2.plot(ref_x, -1.0 * ref_x + 9.5, 'k--', alpha=0.5, label='1/f Pink ( $\beta=1$ )')
ax2.plot(ref_x, -2.0 * ref_x + 8.5, 'k:', alpha=0.5, label='1/f2 Brownian ( $\beta=2$ )')

ax2.set_title("Radial Power Spectral Decay S(f)", fontweight="bold")
ax2.set_xlabel("log10(Normalized Spatial Frequency)")
ax2.set_ylabel("log10(Power Spectral Density)")
ax2.grid(True, linestyle="--", alpha=0.5)
ax2.legend(fontsize=8)

ax3 = fig.add_subplot(1, 3, 3, polar=True)
theta_bins = 36
thetas = np.linspace(0, 2*np.pi, theta_bins, endpoint=False)
for name, color in zip(["AND", "XOR", "ADD", "HYBRID"], ["#e41a1c", "#4daf4a", "#984ea3", "#a65628"]):
    mag = sample_spectra[name]
    h, w = mag.shape
    cy, cx = h // 2, w // 2
    ys, xs = np.indices((h, w))
    ang = (np.arctan2(ys - cy, xs - cx) + np.pi) % (2 * np.pi)

    ang_energy = []
    for t in thetas:
        mask = (ang >= t) & (ang < t + (2*np.pi / theta_bins))
        ang_energy.append(np.sum(mag[mask]))

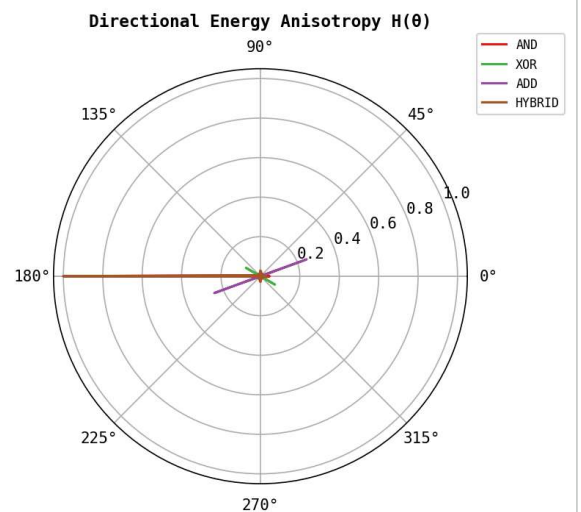
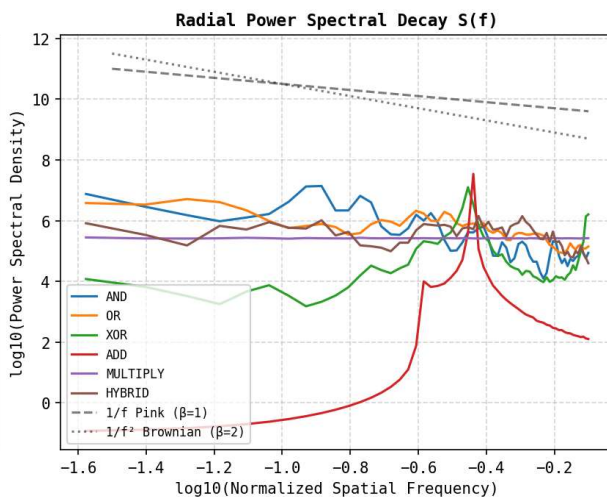
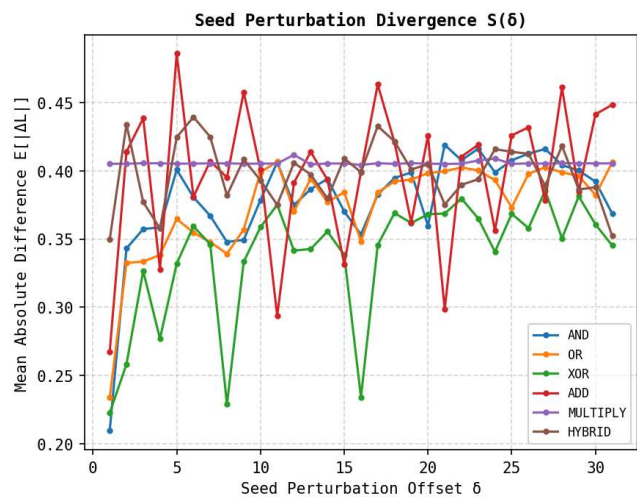
    ang_energy = np.array(ang_energy)
    ang_energy /= (np.max(ang_energy) + 1e-12)
    ang_plot = np.append(ang_energy, ang_energy[0])
    theta_plot = np.append(thetas, thetas[0])
    ax3.plot(theta_plot, ang_plot, label=name, color=color, linewidth=1.5)

ax3.set_title("Directional Energy Anisotropy H( $\theta$ )", fontweight="bold", y=1.08)
ax3.legend(loc='upper right', bbox_to_anchor=(1.25, 1.1), fontsize=8)

plt.tight_layout()
plt.savefig("advanced_spectral_analysis.png", dpi=300, bbox_inches="tight")
plt.show()

print("Exported 'advanced_spectral_analysis.png'")

```



Exported 'advanced_spectral_analysis.png'